Universidad Nacional de Loja
Carrera en Computación
Facultad de Energía las Industrias y los Recursos Naturales no Renovables

Componente Autónomo

Diagramación de punteros y estados de memoria frente a la instanciación de objetos

Integrantes	Matias Encarnación Matias Lavanda Julian Merino Sherman Abarca Jefferson Sarango		
Paralelo	3 A		
Professor	Ing. Cristian Narváez G. Mg.Sc.		
Unidad	1		
Fecha	17/10/2026	**Uso de IA:** Si	**Calificación:**

1. Actividad

Descargar la guía de ejercicios de diagramación publicada en el EVA.

Dibujar a mano (y digitalizar) o diseñar con herramientas vectoriales los estados de la memoria para los casos planteados.

Consolidar las diagramaciones en un informe técnico en formato PDF.

Entregar el archivo bajo el nombre AADiagramaMemoriaApellidosNombres.pdf.

2. Desarrollo

El análisis de la gestión de memoria en entornos de ejecución modernos como el motor V8 de JavaScript exige diferenciar con precisión entre el *Call Stack* y el *Heap*. Además, para la evaluación experimental de este estudio se determinó técnicamente la necesidad de separar los casos en scripts independientes (`Caso1.js`, `Caso2.js` y `Caso3.js`), evitando así interferencias en las mediciones del Recolector de Basura (*Garbage Collector* - GC).

2.1. Estructura de Archivos y Aislamiento de Entorno

Ejecutar la guía en un único script global alteraría los resultados del *Garbage Collector*, debido a que el bucle continuo del Caso 1 interferiría con la pila de eventos e imprecisaría las mediciones del Heap global.

2.2. Análisis Técnico de los Casos de Estudio

2.2.1. Caso 1: Fuga de Memoria por Contexto Léxico Compartido

En el primer escenario se define la función `crearProcesadorResiduos()`, la cual asigna un arreglo masivo de aproximadamente 10 MB en el Heap y mantiene clausuras que retienen referencias indirectas a través del ámbito léxico compartido.

Resultado de Ejecución: El script incrementa progresivamente la memoria del Heap en pasos de 80 MB a 100 MB por segundo, superando los 1123,95 MB hasta requerir una interrupción manual en consola[cite: 3].

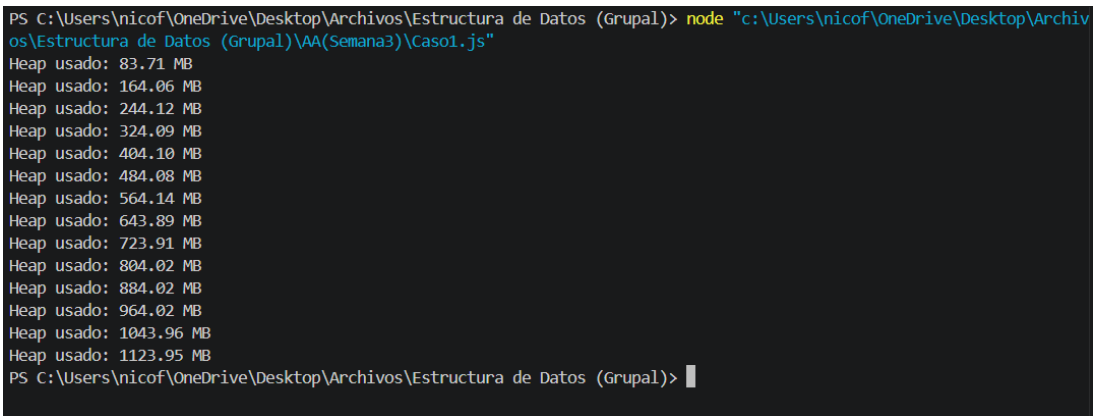


Figura 1: Ejecución en consola del Caso 1 mostrando el incremento continuo del Heap.

2.2.2. Caso 2: Grafo Circular y Referencias Débiles (Map vs. WeakMap)

El segundo escenario analiza la creación de nodos interconectados cíclicamente. Se evalúa el impacto de almacenar la estructura raíz en un `Map` convencional en contraste con el uso potencial de un `WeakMap`.

Resultado de Ejecución: Al correr el comando en consola, el script no genera ninguna salida explícita, comportándose de manera silenciosa mientras procesa los nodos en memoria[cite: 4].

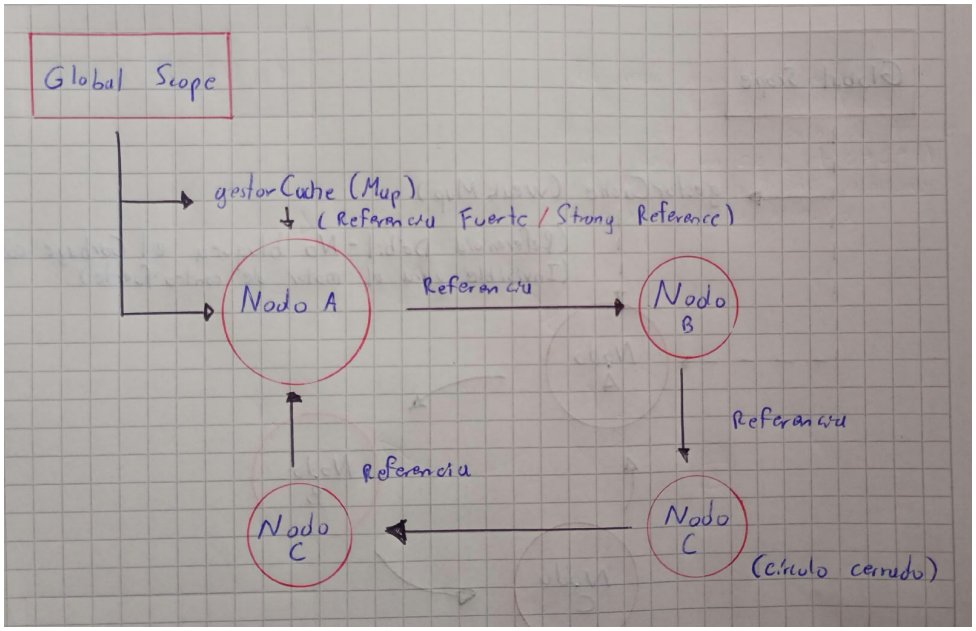


Figura 2: Diagrama de memoria a mano utilizando un Map estándar.

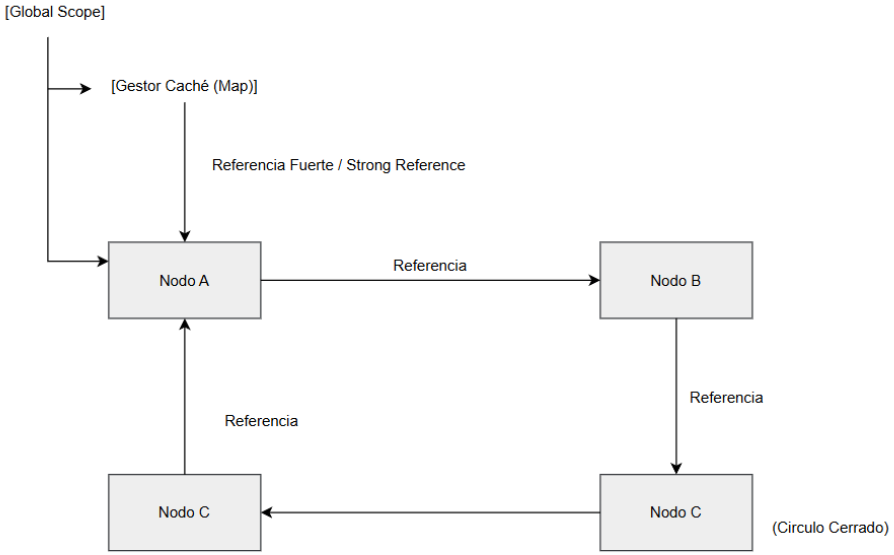


Figura 3: Diagrama de memoria digitalizado utilizando un Map estándar.

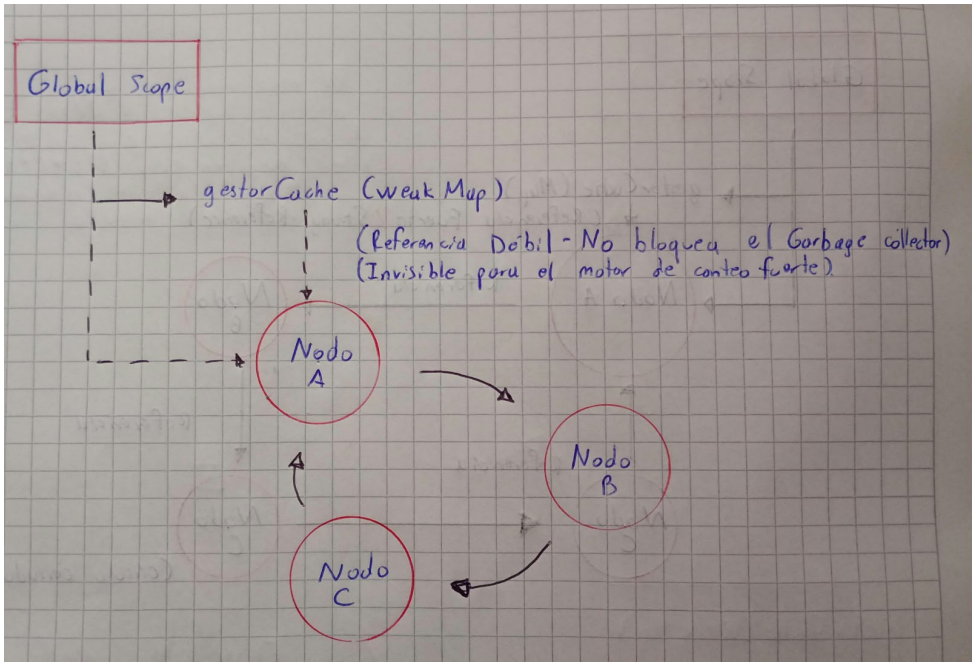


Figura 4: Diagrama de memoria a mano utilizando un WeakMap con referencias débiles.

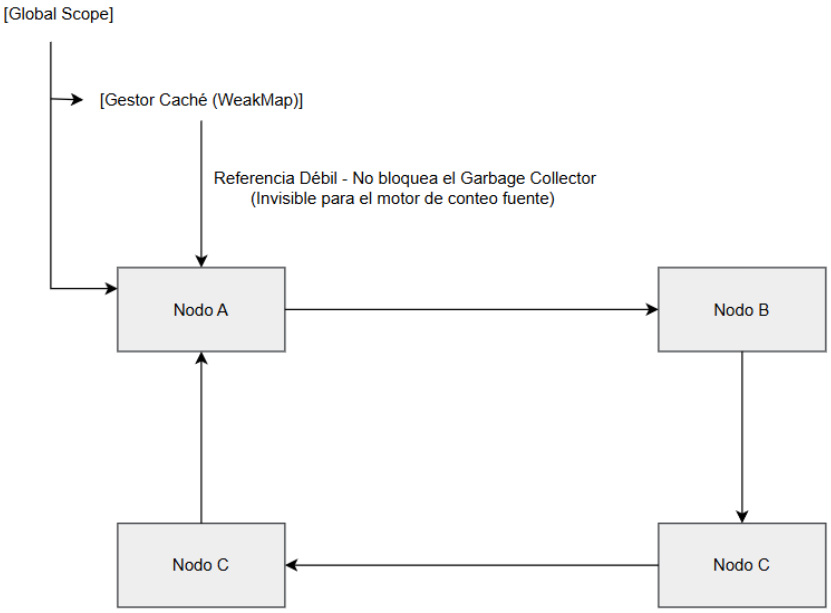


Figura 5: Diagrama de memoria digitalizado utilizando un WeakMap con referencias débiles.

2.2.3. Caso 3: Event Loop Queue y Retención por Callbacks

El tercer caso simula un tráfico de 50 sesiones de usuario donde cada instancia reserva 5 MB mediante un `Uint8Array`. Al no desregistrar los escuchadores del `EventEmitter`, los callbacks retienen los objetos en el Heap[cite: 6, 7].

Resultado de Ejecución: Al emitir el evento global, se despliegan los cierres de sesión y se imprime la advertencia formal de desbordamiento de escuchadores[cite: 6, 7].

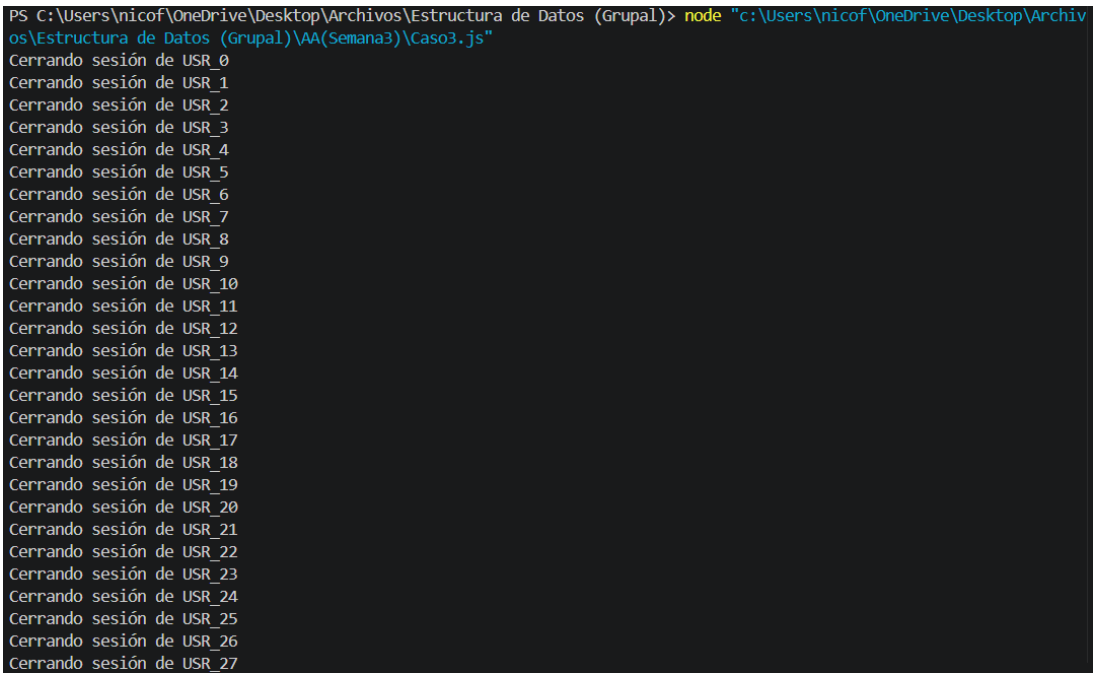


Figura 6: Consola del Caso 3: Cierre secuencial de sesiones de usuario.

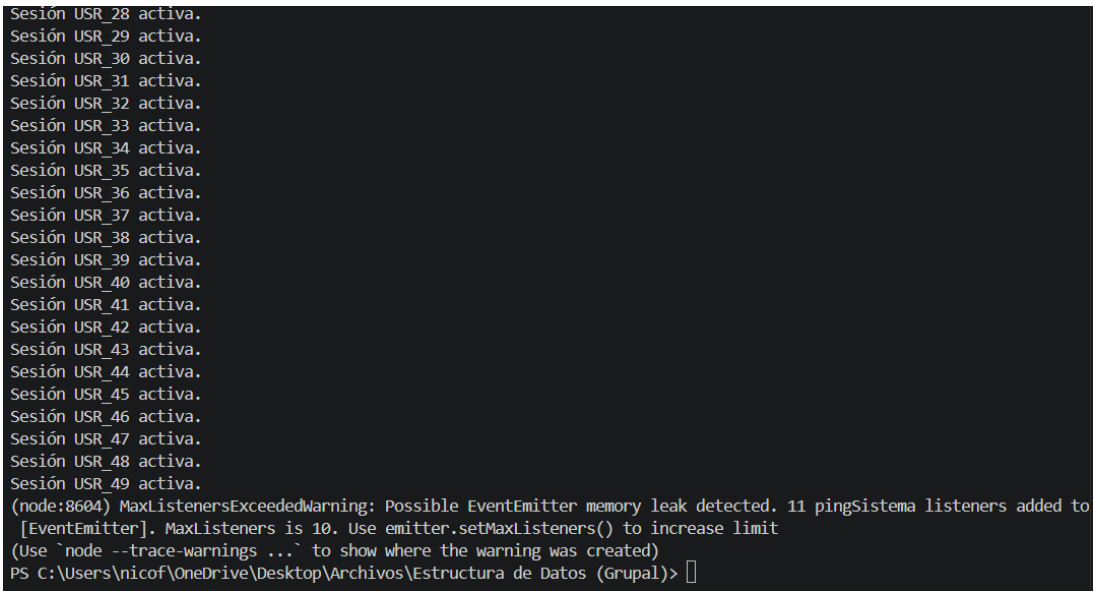


Figura 7: Consola del Caso 3: Activación de sesiones y advertencia de MaxListenersExceeded-Warning.

3. Declaración de uso de IA

Se usó la inteligencia artificial para obtener retroalimentación sobre los puntos importantes de los recursos brindados en la plataforma del EVA. Con base en dicha retroalimentación, se realizó el desarrollo del trabajo, con la ayuda de Gemini 3.8 Flash, utilizando el siguiente prompt:



"De la siguiente información, ¿podrías darme una retroalimentación y resaltar los puntos clave?"

4. Conclusiones

- **C01:** Se comprobó que las fugas de memoria en entornos V8 surgen de la retención implícita de punteros a través de contextos léxicos compartidos y falta de desuscripción de eventos globales.
- **C02:** El recolector de basura opera bajo alcanzabilidad desde la raíz; un grafo circular desconectado de la raíz se libera, pero estructuras retenidas por mapas o eventos persisten en el Heap.
- **C03:** Es fundamental aplicar prácticas defensivas como la eliminación explícita de escuchadores y el uso de colecciones de referencias débiles en arquitecturas eficientes.

NOTA: El documento de ensayo máximo se debe tener una media de cuatro páginas. Con respecto al control anti-plagio será menor al 20 % y del IA del 0 %

Referencias

[1] P. W. Bible, and L. Moser, *An Open Guide to Data Structures and Algorithms*, PALNI Open Press, 2023.

[2] OpenDSA, *Pointers and Memory Management Tutorials*, Virginia Tech, 2021.